

MySQL and JDBC: how the app connects to the database

You own database connectivity and reporting, and you run the connectivity demo. The teacher will test you hard here, so this brief is a script: what to open, what to click, what to say, and the queries that prove it.

1. Your role in one sentence

You explain and demonstrate how the application reaches MySQL: one JNDI datasource and connection pool, used by Hibernate and by hand-written JDBC, with the reports, the database browser and the classroom style JDBC page built on top.

2. What you own

Paths are under `src/main/java/com/campus/placement/` unless shown. Line counts are from `wc -l`.

FILE	LINES	WHAT IT IS
<code>jdbc/PlacementReportDao.java</code>	438	The JDBC layer: JNDI lookup, PreparedStatement, ResultSet, the manual transaction, DatabaseMetaData connection info
<code>jdbc/DatabaseExplorerDao.java</code>	366	Schema and row browsing through DatabaseMetaData, and the guarded read only SQL console
<code>jdbc/BranchReport</code> , <code>CompanyReport</code> , <code>CandidateRow</code>	84, 79, 95	Plain result carriers for the report rows (not JPA entities)
<code>jdbc/ConnectionInfo</code> , <code>ResultTable</code> , <code>TableDetail</code>	45, 98, 208	Carriers for connection facts, console results and table structure
<code>web/ReportsServlet.java</code>	116	Controller for <code>/admin/reports</code> : GET runs the reports, POST closes a drive in a transaction
<code>web/DatabaseExplorerServlet.java</code>	96	Controller for <code>/admin/database</code> : Structure, Rows and Query console tabs
<code>web/ClassroomJdbcServlet.java</code>	84	Controller for <code>/admin/classroom-jdbc</code> : validates input, then calls one of two bean methods
<code>ejb/MockScoreBean.java</code>	113	Stateful bean: <code>addScore</code> (classroom style) and <code>addScoreSafely</code> (pooled, PreparedStatement)
<code>src/main/webapp/WEB-INF/glassfish-resources.xml</code>	57	Defines the connection pool and the JNDI datasource. The only place the URL and password live
<code>views/admin-reports.jsp</code> , <code>admin-database.jsp</code> , <code>admin-classroom.jsp</code>	304, 374, 138	The three pages you demo (under <code>src/main/webapp/WEB-INF/</code>)
<code>src/test/.../jdbc/DatabaseConnectivityTest.java</code> , <code>QueryConsoleGuardTest.java</code>	182, 113	Tests against the real database, and tests of the console guard

Also point at, but do not own: `src/main/resources/META-INF/persistence.xml` (Hibernate reads the same datasource).

3. The 60 second explanation

SAY THIS

Our Java code never contains a database URL or a password. The URL and password sit in one server file, **glassfish-resources.xml**. GlassFish reads it and builds a **connection pool**: four MySQL connections opened up front, growing to at most 24. It publishes that pool under the JNDI name **java:app/jdbc/placementDS**.

The code just asks the naming service for that name and gets a **DataSource**. It calls **getConnection()** and is handed a pooled connection. It runs a **PreparedStatement** with question mark placeholders, reads the **ResultSet**, and **closes** everything, which returns the connection to the pool.

The same datasource is used twice. Hibernate gets it through **persistence.xml** for normal saving and loading. My JDBC code gets it directly for reports, the database browser and a manual transaction. One pool, one set of credentials, two ways of using it.

The one place a URL and password do appear in Java is **MockScoreBean.addScore**, on purpose: it is our copy of the classroom practical, kept so we can compare it with the proper way.

I will show you the chain in the code, then the same rows in the browser, then in MySQL itself.

4. Code walk in VS Code

Open these tabs in this order before you start. Line numbers are current as of this brief, so check them once after any edit.

```
JNDI name → connection pool → DataSource → Connection → PreparedStatement → ResultSet → close (back to the pool)
java:app/jdbc/placementDS → java:app/placementPool → ds.getConnection() → preparedStatement(sql) → executeQuery() → rs.next() → try with resources
```

- 1 WEB-INF/glassfish-resources.xml , lines 38 to 55.** Point at the `jdbc-connection-pool` (line 38) with `steady-pool-size="4"` and `max-pool-size="24"` (lines 41 to 42), then the URL, user and password properties (lines 50 to 52), then the `jdbc-resource` on line 55 that publishes it as `java:app/jdbc/placementDS`.
Say: "This is the only file with the URL and password. It defines the pool and gives it a JNDI name. Change the password here and nothing in Java changes."
- 2 META-INF/persistence.xml , lines 20 and 26.** Point at the Hibernate provider (line 20) and `<jta-data-source>java:app/jdbc/placementDS</jta-data-source>` (line 26).
Say: "Hibernate gets a name, not a URL. It is the same name as the pool above."
- 3 PlacementReportDao.java , lines 66 to 93.** Point at `JNDI_NAME` (66), the `@Resource(lookup = JNDI_NAME)` field (73 to 74), and `lookupDataSource()` (91 to 93) with `new InitialContext().lookup(JNDI_NAME)`.
Say: "The annotation and the explicit lookup do the same thing. I kept the manual one so I can point at the actual JNDI call. The same name feeds Hibernate on the persistence.xml side."
- 4 PlacementReportDao.branchReport() , lines 161 to 180.** Point at the try with resources header (163 to 165): `dataSource().getConnection()`, `connection.prepareStatement(BRANCH_SQL)`, `statement.executeQuery()`. Then the `while (rs.next())` loop (167) reading columns by name. Scroll up to `BRANCH_SQL` (138 to 154) and note the LEFT JOIN.
Say: "Five steps in order: datasource, connection, prepared statement, result set, close. Try with resources closes all three in reverse order even if the query throws. Closing the connection hands it back to the pool."
- 5 PlacementReportDao.searchCandidates() , lines 252 to 299.** Point at `sql.append(" AND s.branch = ? ")` (272) and `parameters.add(branch)` (273), then the binding loop `statement.setObject(i + 1, parameters.get(i))` (297 to 299).
Say: "The SQL text never contains anything a user typed. Only question marks are added. The values are bound afterwards, so they can never be read as SQL."
- 6 PlacementReportDao.closeDriveAndRejectPending() , lines 362 to 419.** Walk down: `setAutoCommit(false)` (378), first `executeUpdate()` (382), second (389), `commit()` (392), `rollback()` in the catch (398), and the `finally` (405 to 418) that restores auto commit and closes. Also show `@TransactionAttribute(NOT_SUPPORTED)` on line 56.
Say: "Closing a drive and rejecting its pending applications must happen together or not at all. Auto commit off, two updates, one commit. If the second update fails, rollback undoes the first. The finally block puts auto commit back before the connection returns to the pool, otherwise the next user would inherit a connection with auto commit off."
- 7 PlacementReportDao.describeConnection() , lines 100 to 121, then DatabaseExplorerDao.describe() , lines 99 to 170.** First method: `connection.getMetaData()` and `meta.getURL()`, `getDriverName()`, `getUserName()`. Second: `getPrimaryKeys` (111), `getColumns` (117), `getImportedKeys` (133), `getIndexInfo` (147).
Say: "DatabaseMetaData lets the connection describe itself and the schema. Nothing on the Reports or Database pages is typed in. It is all asked of the live connection."
- 8 DatabaseExplorerDao.runReadOnly() (216 to 256) and whyRefused() (261 to 281).** Point at the checks in order: no semicolon inside, must start with select, with, show, describe or explain (the pattern on line 62), must not contain a forbidden word (line 67), max 2000 characters. Then in the run: `connection.setReadOnly(true)` (235), `setQueryTimeout(TIMEOUT_SECONDS)` (238), `setMaxRows(MAX_ROWS)` (239).
Say: "Layer one is the keyword guard. Layer two is a read only connection. The rows are capped at 200 and a query is cancelled after 5 seconds. Password hash columns are replaced with (hidden) before they leave this class."
- 9 MockScoreBean.java , addScore (63 to 78) beside addScoreSafely (81 to 91).** First, name the source: **Prac7-MarksEntry** is the practical project, and its `ejb/MarksEntryBean.java` is what `addScore` is copied from. Classroom: `Class.forName` (65), `DriverManager.getConnection(URL, USER, PASSWORD)` (66), `createStatement()` (67), the concatenated query (71 to 72), `return st.executeUpdate(query)` (73), `System.out.println(e)` then `return 0` (75 to 76). Constants with the URL and password are on lines 42 to 45. Safe version: `pooled()` (124 to 132), `? placeholders` (84), `ps.setString / setInt` (85 to 88). Also point at `@Remove finished()` (119 to 122), which the servlet calls after each use.
Say: "Same insert, two styles. The classroom one opens a new connection each call, has the password in the source, builds SQL by joining strings and never closes anything. Unlike the practical, it returns how many rows it saved, so the page can tell the truth. The safe one borrows from the pool and binds values."
- 10 DatabaseConnectivityTest.java , lines 36 to 58 and 95 to 118.** Point at `DriverManager.getConnection(URL, USER, PASSWORD)` (57): a test runs outside the server, so there is no JNDI and no pool. Then `parametersAreNotSql` (97 to 118), which binds `' OR 1=1 --` and asserts zero rows.
Say: "Same SQL, same JDBC calls, only the way the connection is obtained differs. The tests skip themselves if MySQL is down."

5. Live demo plan

Set up before you present

MySQL service MySQL80 running, app running at `http://localhost:8080/placement/`, signed in as `tpo@campus.edu / Campus@2026`. Two windows side by side: the browser and a terminal. Open the mysql client once and leave it open:

```
& "C:\Program Files\MySQL\MySQL Server 8.0\bin\mysql.exe" -u placement_user -p placement_cell
(password: Placement@2026)
```

Or use Workbench with the same user. Every mysql query below is a read unless marked.

- 1 Reports: the live connection.** Go to `http://localhost:8080/placement/admin/reports` (nav link: Reports). The first card, "The live connection", shows Connected and: JNDI name `java:app/jdbc/placementDS`, Database MySQL 8.0.45, JDBC driver MySQL Connector/J 8.4.0, the JDBC URL, Connected as `placement_user@localhost`, Auto commit true, isolation REPEATABLE READ, connection class `com.sun.gjc.spi.jdbc40.ConnectionWrapper40`. Say: "This is not typed into the page. It is read from DatabaseMetaData on a connection borrowed from the pool. The class is a GlassFish wrapper, not MySQL's own, which is proof it came from the pool." Prove it in mysql:

```
SELECT CURRENT_USER(), DATABASE(), @@version;
```

- 2 Reports: the row counts.** Same page, the tiles under the connection card (app_user, student_profile, company, drive, job_application, audit_log). They come from `countRows()`, a plain Statement over a fixed list of table names.

```
SELECT COUNT(*) FROM job_application;
SELECT COUNT(*) FROM student_profile;
SHOW TABLES;
```

The numbers must match the tiles. `SHOW TABLES` lists the same six tables (plus `mock_score` once the classroom page has been used).

- 3 Reports: aggregate SQL.** Scroll to "Placement by branch". Open "The SQL behind this table". Students per branch today: Computer Science 10, Information Technology 6, Electronics 4, Statistics 2, Mechanical 2, Civil 2.

```
SELECT branch, COUNT(*) AS students FROM student_profile GROUP BY branch;
```

Say: "The report is one SQL statement with joins and aggregates. That is why it is JDBC and not the ORM."

- 4 Reports: the prepared search.** "Candidate search": Branch Computer Science, Min CGPA 8.5, click the search button. You get Priya Nair 9.3, Manav Shah 8.9, Aarti Deshpande 8.7. The page shows the filter text with `?` placeholders.

```
SELECT s.roll_no, u.full_name, s.cgpa
FROM student_profile s JOIN app_user u ON u.id = s.user_id
WHERE s.branch = 'Computer Science' AND s.cgpa >= 8.5
ORDER BY s.cgpa DESC;
```

Same three rows.

- 5 SQL injection, proved in mysql.** Concatenated versus bound, side by side. Type these one by one:

```
SELECT COUNT(*) AS concatenated FROM student_profile WHERE branch = '' OR '1'='1';

PREPARE s FROM 'SELECT COUNT(*) AS bound FROM student_profile WHERE branch = ?';
SET @b = '' OR '1'='1';
EXECUTE s USING @b;
DEALLOCATE PREPARE s;
```

Today: `concatenated` returns 26 (every student), `bound` returns 0. The first is what string joining would send. The second is what `setString` does: the payload is only ever data.

Do not type the payload into the Reports branch box and say it reached SQL as a literal. See the warning in section 7.

- 6 **Reports: the transaction.** "Close a drive in one transaction". This changes data. Use **Ironbridge Infrastructure : Graduate Engineer Trainee** (drive id 9), which has no applications at the applied stage, so nothing is lost. Click "Close and reject pending" and confirm. The green message says the drive is closed and 0 pending applications were rejected.

```
SELECT id, status FROM drive WHERE id = 9;
SELECT status, COUNT(*) FROM job_application WHERE drive_id = 9 GROUP BY status;
```

Run the first query before and after: OPEN, then CLOSED. Afterwards reopen the drive from the Drives page (the Open action) so the demo data is as it was. Say: "Two UPDATES, one commit. If the second failed, the first would roll back."

- 7 **Database page: Structure.** Nav: Tools, then Database (/admin/database). In the Tables list on the left click `job_application`, tab Structure. Show the columns, the primary key, the two foreign keys under Relationships, and the unique index `uk_student_drive`.

```
DESCRIBE job_application;
SHOW CREATE TABLE job_application\G
```

Say: "All of this came from DatabaseMetaData: getColumns, getImportedKeys, getIndexInfo. Hibernate created these keys from the JPA mapping. The two views agree."

- 8 **Database page: Rows.** Click the Rows tab, table `app_user`. The `password_hash` column shows (hidden). Then in mysql:

```
SELECT id, email, LEFT(password_hash, 18) AS hash_start FROM app_user LIMIT 2;
```

Say: "The salted PBKDF2 digest is in the table, but the page never returns it. The masking happens in `read()` in the explorer class."

- 9 **Database page: Query console.** Click the Query console tab and run the default query (students by CGPA). Then run the status count below and compare with mysql: 33 REJECTED, 23 APPLIED, 12 SHORTLISTED, 11 INTERVIEW, 6 SELECTED, 85 in all. Then run `DROP TABLE student_profile`: it is refused with "Only SELECT, WITH, SHOW, DESCRIBE and EXPLAIN are allowed here." Try `SELECT 1;` `SELECT 2`: refused as "Only one statement at a time."

```
SELECT status, COUNT(*) FROM job_application GROUP BY status;
```

Say: "The console only reads: keyword guard, read only connection, 200 row cap, 5 second timeout."

- 10 **Proof of the pool.** In mysql:

```
SHOW PROCESSLIST;
```

You will see a few `placement_user` rows in state Sleep, one per pooled connection (the pool starts at 4), plus your own session. Click through Reports and Database again and run it again: the count does not grow with every page.

Say: "These are the pool's connections. Requests borrow and return them. They are not opened per request."

- 11 **Classroom JDBC page.** Nav: Tools, then Classroom JDBC (/admin/classroom-jdbc). The page shows both code samples and a form. First check the table state in mysql:

```
SHOW TABLES LIKE 'mock_score';
```

Before the first classroom save it returns nothing: `addScore` runs `create table if not exists` itself. Save Aarti Deshpande 70, 75, 80 with the method **Classroom (DriverManager and Statement)**. The flash message says it saved and the row appears in "Latest mock scores". Save again with **Pooled (DataSource and PreparedStatement)**. Then in mysql:

```
SELECT * FROM mock_score ORDER BY id DESC;
DESCRIBE mock_score;
SHOW CREATE TABLE mock_score\G
```

Two rows, one from each method.

The next step is the honest injection style demo on the same page.

- 12 Classroom JDBC: what goes wrong with string joining.** On the same page save the name `O'Brien` (scores 70, 75, 80) with the method **Classroom (DriverManager and Statement)**. A red message appears: "The classroom method could not save O'Brien. Its SQL was built by joining strings, and the name changed the statement. The pooled method saves it." No new row shows in "Latest mock scores". The exception is printed to the GlassFish server log (`System.out.println(e)` , line 75): a MySQL syntax error near `Brien` . Now save `O'Brien` again with **Pooled (DataSource and PreparedStatement)**: "Saved O'Brien" and the row appears.

```
SELECT id, sname, aptitude, technical, interview FROM mock_score ORDER BY id DESC;
```

Exactly one `O'Brien` row, the pooled one.

Say: "The apostrophe in the name ended the SQL string early, so the statement changed. That is the everyday face of SQL injection. The bound version treats the same text as plain data."

The servlet checks the name first (letters, spaces and `'` - only), so a full injection payload cannot be typed here. The apostrophe is allowed, and that is enough to show the flaw. Rehearse once. The table `mock_score` exists after any rehearsal; if you want the "table appears" moment on Monday, run `DROP TABLE mock_score;` in mysql first (this table only).

- 13 Optional closer: the tests.** In PowerShell from the project folder run `.\test.ps1` . `DatabaseConnectivityTest` connects to real MySQL with `DriverManager` and checks metadata, the six tables, the injection payload and a rollback. It skips itself if MySQL is down.

6. NetBeans practical vs this project

The practical is the NetBeans project **Prac7-MarksEntry** (`src/java/ejb/MarksEntryBean.java` and `web/index.jsp`). It uses a Stateful bean, the Derby `ClientDriver` , `DriverManager` and a `Statement` . `MockScoreBean` is written in the same shape, pointed at MySQL instead of Derby, and sits beside the real data layer so both can be read side by side. The page `/admin/classroom-jdbc` runs **both** our methods against the same table, `mock_score` , and reads the rows back.

STEP	PRACTICAL: <code>MARKSENTRYBEAN.ADDMARKS</code>	OURS, CLASSROOM: <code>MOCKSCOREBEAN.ADDSCORE</code>	OURS, PROPER: <code>ADDScoresSAFELY</code>
Load driver	<code>Class.forName("org.apache.derby.jdbc.ClientDriver")</code> (line 11)	<code>Class.forName("com.mysql.cj.jdbc.Driver")</code> (65)	Not needed: the container owns the driver
Get connection	<code>DriverManager.getConnection("jdbc:derby://localhost:1527/mydb", "root", "root")</code> (12), a new connection every call	<code>DriverManager.getConnection(URL, USER, PASSWORD)</code> (66), a new TCP connection every call	<code>pooled()</code> (124 to 132): JNDI lookup, <code>ds.getConnection()</code> from the pool
URL and password	Written inside the method	Constants, lines 42 to 45	Nowhere in Java. In <code>glassfish-resources.xml</code>
Statement	<code>con.createStatement()</code> (13)	<code>con.createStatement()</code> (67)	<code>PreparedStatement</code> (83 to 84)
Build the SQL	<code>"insert into marks(sname,marks1,marks2,marks3) values('" + sname + "', " + m1 ...</code> (14 to 15)	Same joining, into <code>mock_score</code> (71 to 72)	<code>values(?, ?, ?, ?)</code> then <code>setString</code> , <code>setInt</code> (85 to 88)
Run it	<code>st.executeUpdate(query)</code> (16), result thrown away, method is <code>void</code>	<code>return st.executeUpdate(query)</code> (73): returns rows inserted	<code>ps.executeUpdate()</code> (89)
Close	Never: no <code>close()</code> on the statement or the connection	Never: same leak	Try with resources closes the statement and returns the connection
Errors	<code>catch (Exception e) { System.out.println(e); }</code> (17 to 19). <code>index.jsp</code> then prints "Marks entered successfully" whatever happened	Still prints (75), but returns 0 (76). The servlet shows an error message instead of "Saved"	<code>throws SQLException</code> : the servlet shows "The score could not be saved"
Injection	Vulnerable: the name goes straight into the SQL, and the JSP does no validation	Vulnerable by design. A quote in the name changes the SQL. The servlet allows only letters and <code>. ' -</code> in the name (lines 58 to 59), and <code>O'Brien</code> shows the flaw	Safe: values are da
Who calls it	<code>index.jsp</code> , through a static bean field set in <code>jspInit</code> with a global JNDI lookup	<code>ClassroomJdbcServlet</code> , looking the bean up on each request (line 91) and calling <code>finished()</code> (<code>@Remove</code>) after	Same servlet

How to talk through it

"The practical is the right way to learn the five JDBC steps. It is the wrong way to ship them. Ours does the same insert both ways so you can see what changes: where the connection comes from, how the SQL is built, who closes what, and whether the caller ever learns it failed." Point out that `ClassroomJdbcServlet` validates the name and the 0 to 100 range before either method runs (lines 58 to 61), and looks the bean up by JNDI on every request (line 90) because a Stateful bean is one instance per client and a servlet is shared.

7. Questions you will be asked

CAREFUL

Do not claim the Reports branch box proves injection safety. It is a dropdown, and `ReportsServlet` (line 56) uses `Branches.isKnown`: an unknown value is dropped and the search runs with no branch filter. The protection that matters is the `?` placeholder in the DAO. Prove that with the mysql `PREPARE` demo (step 5) and the test `parametersAreNotSql`. For an in-app example use the O'Brien step on the Classroom JDBC page.

What happens if the password in the pool is wrong?

MySQL refuses the login with "Access denied for user placement_user". The password lives in one place, `glassfish-resources.xml` (line 52). The pool is set with `ping="true"`, so GlassFish tests it and reports the failure. In the app, `describeConnection()` catches the `SQLException` (lines 115 to 119) and the Reports card shows "Unavailable" with MySQL's message instead of crashing. Pages that go through Hibernate will also fail. Fix the password in the XML and redeploy with `.\run.ps1`. No Java file changes.

What is a connection pool?

A set of already open, already authenticated MySQL connections kept by the server. Code borrows one with `getConnection()` and gives it back with `close()`. Ours (lines 41 to 47): 4 connections opened to start, at most 24, idle ones dropped after 300 seconds, and each is validated before it is handed out.

Why not open a connection per request?

Opening a TCP connection and logging in to MySQL costs far more than the query itself, and MySQL allows only a limited number of connections. A pool pays that cost once and reuses. That is exactly what `MockScoreBean.addScore` does wrong: it opens a new connection on every call and never closes it.

Is this safe from SQL injection? Show me.

Yes for values. Show `searchCandidates` lines 271 to 299: the SQL only ever gets `?` added, and values are bound with `setObject`. Then run the mysql `PREPARE` demo: concatenated returns 26 rows, bound returns 0. Two honest limits. A table name cannot be a `?`, so `requireKnownTable` (explorer, line 347) accepts only names the database itself reported, and `countRows` (report DAO, 426 to 437) uses a fixed list. And the classroom method is unsafe on purpose, which is why the servlet validates first.

What goes wrong with the classroom version?

Four things, all visible in `addScore`. It opens a new connection on every call and never closes it, so connections leak. The URL and password are in the source. The SQL is built by joining strings, so a name like `O'Brien` ends the string early and the statement is rejected, which is SQL injection in its everyday form. And errors are only printed: the practical hides the failure completely, ours returns 0 so the page shows an error, but the exception itself still only goes to the server log. The pooled method saves the same name because the value is bound, not joined.

What is the difference between Statement and PreparedStatement?

A Statement sends the finished SQL text. A PreparedStatement sends the SQL with `?` placeholders, and the values travel separately, typed, so they can never be read as SQL. It also lets the database reuse the parsed statement. We still use a plain Statement in a few places where no user value is joined in: row counts over a fixed or database-reported table list, and the console, which runs officer typed reads behind the guard.

What if you forget to close the connection?

With a pool, `close()` does not close the socket, it returns the connection. Forget it and the connection is never returned. After 24 leaks the pool is empty and every request waits. That is why every connection here is in a try with resources block, and why `closeDriveAndRejectPending` closes in a `finally`.

Why JDBC for reports if you already have Hibernate?

Hibernate is right for loading an object, changing a field and saving it. Reports aggregate across tables and return numbers, never written back. Plain SQL with `COUNT`, `GROUP BY` and a `LEFT JOIN` is clearer than forcing it into JPQL. Both use the same datasource, so it is one pool, one set of credentials.

Explain the transaction in closeDriveAndRejectPending.

Two updates must succeed together: set the drive to CLOSED, and set its APPLIED applications to REJECTED. `setAutoCommit(false)`, run both, `commit()`. On any `SQLException`, `rollback()` undoes both. The finally block restores auto commit, then closes. The bean is `NOT_SUPPORTED` so the container does not start its own JTA transaction, which would conflict with one driven by hand.

Can someone destroy data through the SQL console?

Three layers. One: the guard (must start with select, with, show, describe or explain, one statement, no forbidden keyword such as drop, update or into). Two: `setReadOnly(true)` on the connection, so the server refuses writes even if the guard were fooled. Three: the database user only has rights on `placement_cell`. Be honest about layer three: that user does have all privileges on this schema, so layers one and two are what stop writes here. Rows are capped at 200 and a query is cancelled after 5 seconds.

Why is the MySQL driver not inside the WAR? Do you still need Class.forName?

The pool is created by GlassFish before the application loads, so the driver must be on the server's classpath: `run.ps1` copies it into `domain1\lib`. And `Class.forName` is not needed with a modern JDBC 4 driver, which registers itself. It appears in `MockScoreBean.addScore` only because the practical uses it.

Where is DatabaseMetaData used?

Two places. `describeConnection()` (report DAO, 100 to 121) fills the "live connection" card: product name and version, driver, URL, user, isolation. `describe()` (explorer DAO, 99 to 170) reads columns, primary keys, foreign keys and indexes for the Structure tab. `tableNames()` uses `getTables` as the whitelist.

The password is in plain text in an XML file. Is that acceptable?

It is a local database made for this project and the user only has rights on one schema. It is listed as an accepted risk in `docs/SECURITY.md` (item A1). The production step would be a GlassFish password alias or a secret from the environment. The point of the design is that it is in one server file, not compiled into classes.

8. If something goes wrong

SYMPTOM	FIX
Reports card says "Unavailable", or pages show a database error	MySQL is probably stopped. In an admin PowerShell: <code>Start-Service MySQL80</code> . Reload the page. No restart of the app needed.
<code>http://localhost:8080/placement/</code> does not open	The server is down. From the project folder run <code>.\run.ps1</code> and wait for it to deploy. Do not use <code>-Reseed</code> .
Port 8080 already in use	An old server is running. Run <code>.\stop.ps1</code> , then <code>.\run.ps1</code> .
Redirected to the login page	Session expired. Sign in again as <code>tpo@campus.edu</code> . A student account gets a 403 on <code>/admin/</code> pages.
Access denied for user <code>placement_user</code>	Wrong password in <code>glassfish-resources.xml</code> , or the user was not created. Correct password is <code>Placement@2026</code> . Recreate the user with the CREATE USER and GRANT lines in <code>README.md</code> , then <code>.\run.ps1</code> .
<code>mysql</code> is not recognised	Use the full path: <code>& "C:\Program Files\MySQL\MySQL Server 8.0\bin\mysql.exe" -u placement_user -p placement_cell</code> , or use Workbench.
Classroom page shows no scores	Normal. The table <code>mock_score</code> is created by the first classroom method save. <code>SHOW TABLES</code> will confirm.
A POST is rejected (form does nothing or an error page)	The CSRF token went stale. Reload the page and submit again.
Console says a query is refused	Read the reason. It only allows one SELECT, WITH, SHOW, DESCRIBE or EXPLAIN. Words like set, do, use and into are blocked wherever they appear.
Row counts differ from what you rehearsed	Someone applied, closed a drive or saved a score. Re-run your mysql query live; the browser and mysql should always agree with each other.

Written by and for the team. Keep this open in a second window while you present.